

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ № 1

з дисципліни “Професійний Python”

на тему “Організація професійного Python-проекту та Git”

Виконав

студент 3 курсу

групи ФЕП-33

Рак Богдан Ігорович

Перевірив

Професор кафедри ОІТ

Свелеба Сергій Андрійович

Львів – 2026

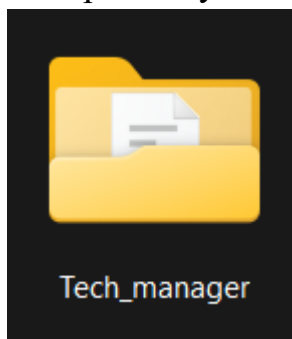
Мета: Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл pyproject.toml, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

1. Встановити Python.

```
C:\Users\w1nzeen>py
Python 3.13.15 (tags/v3.13.15:4061bc4, Aug  5 2026, 13:05:39) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

2. Створити каталог проєкту.

Створив теку для проєкту



3. Створити .venv, 4. Активувати virtual environment.

```
PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> python -m venv .venv
PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> .\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> |
```

Створив віртуальне середовище та активував його.

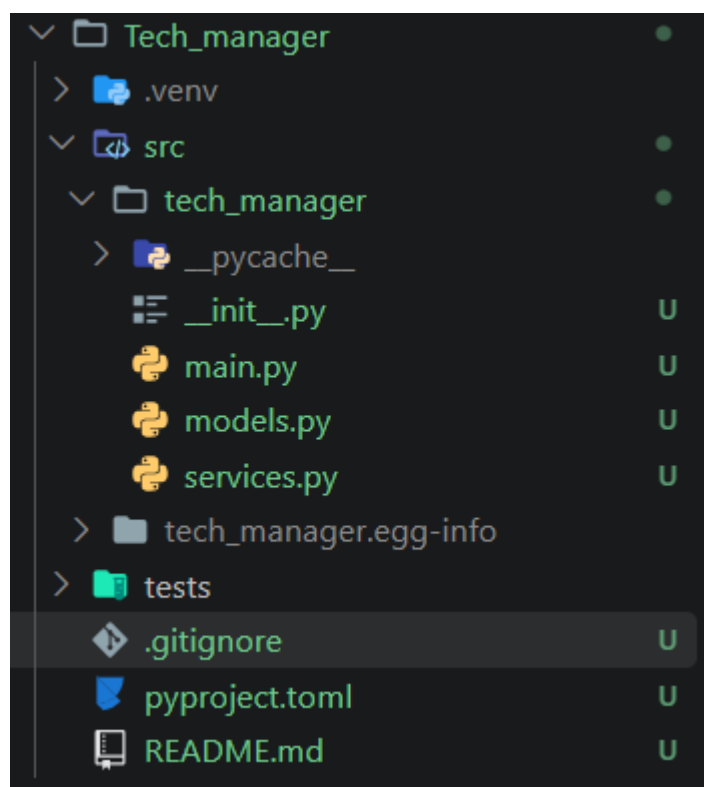
5. Ініціалізувати Git repository.

```
(.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> git init
Initialized empty Git repository in C:/Users/w1nzeen/OneDrive/Робочий стіл/SI 3 term/prof python/Tech_manager/.git/
(.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> |
```

6. Створити .gitignore.

```
.gitignore U X
prof python > Tech_manager > .gitignore
1  .venv/
2  __pycache__/
3  *.pyc
4  .pytest_cache/
5  .idea/
6  .vscode/
7  dist/
8  build/
9  *.egg-info/
10 .env
11 |
```

7. Створити src-layout.



8. Створити package.

```
(.venv) PS C:\Users\winzeen\OneDrive\Робочий стіл\SI 3 term\prof python\tech_manager> python -m pip install -e .
Obtaining file:///C:/Users/winzeen/OneDrive/%D0%A0%D0%BE%D0%B1%D0%BE%D1%87%D0%B8%D0%B9%20%D1%81%D1%82%D1%96%D0%BB/SI%203%20term/prof%20python/Tec
h_manager
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: tech-manager
Building editable for tech-manager (pyproject.toml) ... done
Created wheel for tech-manager: filename=tech_manager-0.1.0-0.editable-py3-none-any.whl size=1604 sha256=679853bcf2cbb3d10216176901d0917e8b1ee8
357d23364b133280949691c73e
Stored in directory: C:\Users\winzeen\AppData\Local\Temp\pip-ephem-wheel-cache-ee6aeb3u\wheels\8c\24\d0\d35d8f318125127c82fe56d5fdae0c8d95f1dfa
f8582fed618
Successfully built tech-manager
Installing collected packages: tech-manager
Attempting uninstall: tech-manager
Found existing installation: tech-manager 0.1.0
Uninstalling tech-manager-0.1.0:
Successfully uninstalled tech-manager-0.1.0
Successfully installed tech-manager-0.1.0
(.venv) PS C:\Users\winzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager>
```

9. Створити models.

```
models.py U X
prof python > Tech_manager > src > tech_manager > models.py > Tech > name
1 from dataclasses import dataclass
2
3 @dataclass
4 class Tech:
5     inventory_number: str
6     type: str
7     vendor: str
8     model: str
9     price: float
10
11     @property
12     def name(self) -> str:
13         return f"{self.type}: {self.vendor} {self.model}"
```

10. Створити services.py.

```
services.py U X
prof python > Tech_manager > src > tech_manager > services.py > price_of_all_tech
1 from tech_manager.models import Tech
2
3 def search_by_model(tech_list: list[Tech], model: str) -> list[Tech]:
4     return [tech for tech in tech_list if tech.model.lower() == model.lower()]
5
6
7 def sort_by_vendor(tech_list: list[Tech]) -> list[Tech]:
8     return sorted(tech_list, key=lambda tech: tech.vendor.lower())
9
10
11 def avg_price(tech_list: list[Tech]) -> float:
12     if not tech_list:
13         return 0.0
14     total_price = sum(tech.price for tech in tech_list)
15     return total_price / len(tech_list)
16
17 def find_most_expensive_tech(tech_list: list[Tech]) -> Tech:
18     if not tech_list:
19         return None
20     return max(tech_list, key=lambda tech: tech.price)
21
22 def filter_by_price_range(tech_list: list[Tech], min_price: float, max_price: float) -> list[Tech]:
23     return [tech for tech in tech_list if min_price <= tech.price <= max_price]
24
25 def price_of_all_tech(tech_list: list[Tech]) -> float:
26     return sum(tech.price for tech in tech_list)
```

11. Створити main.py.

```
main.py U x
prof python > Tech_manager > src > tech_manager > main.py > main
1 from tech_manager.models import Tech
2 from tech_manager.services import (search_by_model,
3                                   sort_by_vendor,
4                                   avg_price,
5                                   find_most_expensive_tech,
6                                   filter_by_price_range,
7                                   price_of_all_tech)
8
9 def create_demo_tech_list() -> list[Tech]:
10     return [
11         Tech(inventory_number="001", type="Laptop", vendor="Dell", model="XPS 13", price=999.99),
12         Tech(inventory_number="002", type="Smartphone", vendor="Apple", model="iPhone 13", price=799.99),
13         Tech(inventory_number="003", type="Tablet", vendor="Samsung", model="Galaxy Tab S7", price=649.99),
14         Tech(inventory_number="004", type="Laptop", vendor="HP", model="Spectre x360", price=1199.99),
15         Tech(inventory_number="005", type="Smartphone", vendor="Google", model="Pixel 6", price=599.99),
16     ]
17
18 def print_tech_list(tech_list: list[Tech]) -> None:
19     for tech in tech_list:
20         print(f"Inventory Number: {tech.inventory_number}, Type: {tech.type}, Vendor: {tech.vendor}, Model: {tech.model}, Price: $")
21
22
```

12. Реалізувати свій індивідуальний варіант.

Оскільки я 13 в списку, я реалізував 13 варіант.

Варіант 13. Система обліку комп'ютерної техніки

Для пристрою:

- інвентарний номер;
- тип;
- виробник;
- модель;
- вартість.

Реалізувати:

- пошук;
- фільтрацію за виробником;
- середню вартість;
- найдорожчий пристрій;
- загальну вартість обладнання.

13. Додати type hints.

```
prof python > Tech_manager > src > tech_manager > services.py > price_of_all_tech
1  from tech_manager.models import Tech
2
3  def search_by_model(tech_list: list[Tech], model: str) -> list[Tech]:
4      return [tech for tech in tech_list if tech.model.lower() == model.lower()]
5
6
7  def sort_by_vendor(tech_list: list[Tech]) -> list[Tech]:
8      return sorted(tech_list, key=lambda tech: tech.vendor.lower())
9
10
11 def avg_price(tech_list: list[Tech]) -> float:
12     if not tech_list:
13         return 0.0
14     total_price = sum(tech.price for tech in tech_list)
15     return total_price / len(tech_list)
16
17 def find_most_expensive_tech(tech_list: list[Tech]) -> Tech:
18     if not tech_list:
19         return None
20     return max(tech_list, key=lambda tech: tech.price)
21
22 def filter_by_price_range(tech_list: list[Tech], min_price: float, max_price: float) -> list[Tech]:
23     return [tech for tech in tech_list if min_price <= tech.price <= max_price]
24
25 def price_of_all_tech(tech_list: list[Tech]) -> float:
26     return sum(tech.price for tech in tech_list)
```

Додав type hints для функцій.

14. Створити pyproject.toml.

```
pyproject.toml
prof python > Tech_manager > pyproject.toml
1  [build-system]
2  requires = ["setuptools>=70"]
3  build-backend = "setuptools.build_meta"
4
5  [project]
6  name = "tech-manager"
7  version = "0.1.0"
8  description = "Tech management laboratory project"
9  requires-python = ">=3.11"
10
11 [project.scripts]
12 Tech-manager = "tech_manager.main:main"
13
```

15. Встановити package у editable mode.

```
(.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> python -m pip install -e .
Obtaining file:///C:/Users/w1nzeen/OneDrive/%D0%A0%D0%BE%D0%B1%D0%BE%D1%87%D0%B8%D0%B9%20%D1%81%D1%82%D1%96%D0%BB/SI%203%20term/prof%20python/Tec
h_manager
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: tech-manager
Building editable for tech-manager (pyproject.toml) ... done
Created wheel for tech-manager: filename=tech_manager-0.1.0-0.editable-py3-none-any.whl size=1604 sha256=e20fdc7c0a3af7aab68f9d8a3a4f7a61519c57
9bf5d0a5392773d71e41e0fa50
Stored in directory: C:\Users\w1nzeen\AppData\Local\Temp\pip-ephem-wheel-cache-4c9ehi1g\wheels\8c\24\d0\d35d8f318125127c82fe56d5fdae0c8d95f1dfa
f8582fed618
Successfully built tech-manager
Installing collected packages: tech-manager
Attempting uninstall: tech-manager
Found existing installation: tech-manager 0.1.0
Uninstalling tech-manager-0.1.0:
Successfully uninstalled tech-manager-0.1.0
Successfully installed tech-manager-0.1.0
(.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager>
```

16. Перевірити запуск.

```
(.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> tech-manager

Tech Manager Menu:
1. View all tech items
2. Search by model
3. Sort by vendor
4. Calculate average price
5. Find the most expensive tech item
6. Filter by price range
7. Calculate total price of all tech items
8. Exit
Enter your choice: 1

All Tech Items:
Inventory Number: 001, Type: Laptop, Vendor: Dell, Model: XPS 13, Price: $999.99
Inventory Number: 002, Type: Smartphone, Vendor: Apple, Model: iPhone 13, Price: $799.99
Inventory Number: 003, Type: Tablet, Vendor: Samsung, Model: Galaxy Tab S7, Price: $649.99
Inventory Number: 004, Type: Laptop, Vendor: HP, Model: Spectre x360, Price: $1199.99
Inventory Number: 005, Type: Smartphone, Vendor: Google, Model: Pixel 6, Price: $599.99

Tech Manager Menu:
1. View all tech items
2. Search by model
3. Sort by vendor
4. Calculate average price
5. Find the most expensive tech item
6. Filter by price range
7. Calculate total price of all tech items
8. Exit
Enter your choice: 2
Enter the model to search for: Galaxy Tab S7

Search Results:
Inventory Number: 003, Type: Tablet, Vendor: Samsung, Model: Galaxy Tab S7, Price: $649.99
```

17. Створити README.md.

```
prof python > Tech_manager > README.md > abc # Tech Manager > abc ## Description
1  # Tech Manager
2
3  Laboratory project for the course
4  "Professional Python".
5
6  ## Description
7
8  Console application for managing technic.
9
10 ## Requirements
11
12 Python 3.11+
13
14 ## Installation
15
16 python -m venv .venv
17
18 Activate the virtual environment.
19
20 python -m pip install -e .
21
22 ## Run
23
24 python -m tech_manager.main
25
26 or
27
28 tech-manager
29
30 ## Project structure
31
32 src/tech_manager/models.py
33 Data models.
34
35 src/tech_manager/services.py
36 Business logic.
```

18. Зробити не менше трьох Git commits.

```
• (.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> git log --oneline
5ca2695 (HEAD -> main, origin/main) Created .toml file
513968d Created .gitignore file
59d0217 Created README file
f6bfc80 Created init file
3257f66 Created services file
d12f10f Created models file
f432e2e Created main file
○ (.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager>
```

19.Перевірити git status.

```
• (.venv) PS C:\Users\winzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
○ (.venv) PS C:\Users\winzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> |
```

Висновок: Виконуючи цю лабораторну роботу, можна ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл pyproject.toml, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

Додаток:

decorators.py

```
from functools import wraps
```

```
from time import perf_counter
```

```
def measure_time(func):
```

```
    @wraps(func)
```

```
    def wrapper(*args, **kwargs):
```

```
        start = perf_counter()
```

```
        result = func(*args, **kwargs)
```

```
        elapsed = perf_counter() - start
```

```
        print(f"[measure_time] {func.__name__}: {elapsed:.8f} s")
```

```
        return result
```

```
    return wrapper
```

```
def repeat(count: int):
```

```
def decorator(func):  
    @wraps(func)  
    def wrapper(*args, **kwargs):  
        result = None  
        for _ in range(count):  
            result = func(*args, **kwargs)  
        return result  
  
    return wrapper  
  
return decorator
```

main.py

```
from tech_manager.decorators import repeat  
from tech_manager.models import TECH_TYPES, Tech  
from tech_manager.services import (  
    average_of_values,  
    avg_price,  
    benchmark_search,  
    build_summary_report,  
    count_by_type,  
    count_by_vendor,  
    create_price_filter,  
    create_tech_index,  
    create_tech_record,
```

```
filter_by_price_range,  
filter_items,  
find_cheapest_tech,  
find_most_expensive_tech,  
get_search_history,  
get_unique_types,  
get_unique_vendors,  
group_by_type,  
group_by_vendor_and_type,  
price_of_all_tech,  
process_pipeline,  
search_by_model,  
sort_by_vendor,  
sort_items,  
)
```

```
def create_demo_tech_list() -> list[Tech]:
```

```
    return [  
        Tech(inventory_number="001", type="Laptop", vendor="Dell", model="XPS  
13", price=999.99),  
        Tech(inventory_number="002", type="Smartphone", vendor="Apple",  
model="iPhone 13", price=799.99),  
        Tech(inventory_number="003", type="Tablet", vendor="Samsung",  
model="Galaxy Tab S7", price=649.99),  
        Tech(inventory_number="004", type="Laptop", vendor="HP", model="Spectre  
x360", price=1199.99),  
        Tech(inventory_number="005", type="Smartphone", vendor="Google",  
model="Pixel 6", price=599.99),  
        Tech(inventory_number="006", type="Monitor", vendor="Dell",  
model="UltraSharp U2720Q", price=549.99),
```

```

    Tech(inventory_number="007", type="Laptop", vendor="Apple",
model="MacBook Air M2", price=1099.00),

    Tech(inventory_number="008", type="Printer", vendor="HP", model="LaserJet
Pro", price=249.99),

]

```

```

def print_tech_list(tech_list: list[Tech]) -> None:

```

```

    for tech in tech_list:

```

```

        print(

```

```

            f'Inventory Number: {tech.inventory_number}, Type: {tech.type}, "

```

```

            f'Vendor: {tech.vendor}, Model: {tech.model}, Price: ${tech.price:.2f}'

```

```

        )

```

```

def print_menu() -> None:

```

```

    print("\nTech Manager Menu:")

```

```

    print(" 1. View all tech items")

```

```

    print(" 2. Search by model")

```

```

    print(" 3. Sort by vendor")

```

```

    print(" 4. Sort by price (universal sort)")

```

```

    print(" 5. Calculate average price")

```

```

    print(" 6. Find the most expensive / cheapest tech item")

```

```

    print(" 7. Filter by price range")

```

```

    print(" 8. Filter by price using a closure (price >= threshold)")

```

```

    print(" 9. Unique vendors and unique types")

```

```

    print("10. Group by type / group by vendor and type (nested)")

```

```

    print("11. Count items by type and by vendor (Counter)")

```

```

    print("12. Build tech index and search by inventory number")

```

```

    print("13. Total price of all tech items / summary report")

```

```

    print("14. Benchmark: list search vs dict search")

```

```
print("15. Demo: *args, **kwargs, pipeline, decorators, search history")
print("16. Exit")
```

```
def run_args_kwargs_pipeline_demo(tech_list: list[Tech]) -> None:
    demo_average = average_of_values(999.99, 799.99, 649.99, 1199.99)
    print(f"\nAverage via *args: ${demo_average:.2f}")
```

```
new_tech = create_tech_record(
    inventory_number="009",
    type="Tablet",
    vendor="Lenovo",
    model="Tab P11",
    price=349.99,
)
print(f"Created via **kwargs: {new_tech}")
```

```
cheap_laptops = process_pipeline(
    tech_list,
    lambda items: filter_items(items, create_price_filter(0)),
    lambda items: [tech for tech in items if tech.type == "Laptop"],
    lambda items: sort_items(items, key=lambda tech: tech.price),
)
print("\nPipeline result (laptops sorted by price):")
print_tech_list(cheap_laptops)
```

```
@repeat(3)
def show_message() -> None:
    print("Processing...")
```

```
print("\nParameterized decorator demo (@repeat(3)):")
```

```
show_message()
```

```
print("\nAllowed tech types (tuple):", TECH_TYPES)
```

```
print("Recent search history (deque snapshot):", get_search_history())
```

```
def main() -> None:
```

```
    tech_list = create_demo_tech_list()
```

```
    choice = 0
```

```
    while choice != 16:
```

```
        print_menu()
```

```
        choice = int(input("Enter your choice: "))
```

```
        if choice == 1:
```

```
            print("\nAll Tech Items:")
```

```
            print_tech_list(tech_list)
```

```
        elif choice == 2:
```

```
            model = input("Enter the model to search for: ")
```

```
            search_results = search_by_model(tech_list, model)
```

```
            print("\nSearch Results:")
```

```
            print_tech_list(search_results)
```

```
        elif choice == 3:
```

```
            sorted_tech = sort_by_vendor(tech_list)
```

```
            print("\nSorted by Vendor:")
```

```

    print_tech_list(sorted_tech)
elif choice == 4:
    sorted_tech = sort_items(tech_list, key=lambda tech: tech.price)
    print("\nSorted by Price:")
    print_tech_list(sorted_tech)
elif choice == 5:
    average_price = avg_price(tech_list)
    print(f"\nAverage Price of Tech Items: ${average_price:.2f}")
elif choice == 6:
    most_expensive = find_most_expensive_tech(tech_list)
    cheapest = find_cheapest_tech(tech_list)
    if most_expensive:
        print(f"\nMost Expensive: {most_expensive.name} -
    ${most_expensive.price:.2f}")
    if cheapest:
        print(f"\nCheapest: {cheapest.name} - ${cheapest.price:.2f}")
elif choice == 7:
    min_price = float(input("Enter minimum price: "))
    max_price = float(input("Enter maximum price: "))
    filtered_tech = filter_by_price_range(tech_list, min_price, max_price)
    print("\nFiltered by Price Range:")
    print_tech_list(filtered_tech)
elif choice == 8:
    threshold = float(input("Enter minimum price threshold: "))
    is_above_threshold = create_price_filter(threshold)
    filtered_tech = filter_items(tech_list, is_above_threshold)
    print(f"\nItems with price >= ${threshold:.2f}:")
    print_tech_list(filtered_tech)
elif choice == 9:

```

```

print("\nUnique vendors:", get_unique_vendors(tech_list))

print("Unique types:", get_unique_types(tech_list))

elif choice == 10:

    grouped = group_by_type(tech_list)

    print("\nGrouped by type:")

    for tech_type, items in grouped.items():

        print(f" {tech_type} -> {len(items)} item(s)")

    nested = group_by_vendor_and_type(tech_list)

    print("\nGrouped by vendor and type (nested):")

    for vendor, types_dict in nested.items():

        print(f" {vendor}:")

        for tech_type, items in types_dict.items():

            print(f" {tech_type} -> {len(items)} item(s)")

elif choice == 11:

    print("\nCount by type:", count_by_type(tech_list))

    print("Count by vendor:", count_by_vendor(tech_list))

elif choice == 12:

    index = create_tech_index(tech_list)

    inventory_number = input("Enter inventory number to look up: ")

    found = index.get(inventory_number)

    print("\nSearch by inventory number:", found if found else "Not found")

elif choice == 13:

    total_price = price_of_all_tech(tech_list)

    print(f"\nTotal Price of All Tech Items: ${total_price:.2f}")

    print("Summary report:", build_summary_report(tech_list))

elif choice == 14:

    inventory_number = input("Enter inventory number to benchmark search for: ")

    timings = benchmark_search(tech_list, inventory_number)

```

```

        print(f"\nList search: {timings['list_search_seconds']:.8f} s")
        print(f"Dict search: {timings['dict_search_seconds']:.8f} s")
    elif choice == 15:
        run_args_kwargs_pipeline_demo(tech_list)
    elif choice == 16:
        print("Exiting the program.")
    else:
        print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()

```

models.py

```

from dataclasses import dataclass

```

```

TECH_TYPES: tuple[str, ...] = ("Laptop", "Smartphone", "Tablet", "Monitor",
"Printer")

```

```

@dataclass

```

```

class Tech:

```

```

    inventory_number: str

```

```

    type: str

```

```

    vendor: str

```

```

    model: str

```

```

    price: float

```

```

    @property

```

```

    def name(self) -> str:

```

```
return f"{self.type}: {self.vendor} {self.model}"
```

services.py

```
from collections import Counter, defaultdict, deque
```

```
from collections.abc import Callable
```

```
from time import perf_counter
```

```
from tech_manager.decorators import measure_time
```

```
from tech_manager.models import Tech
```

```
_search_history: deque[str] = deque(maxlen=5)
```

```
def search_by_model(tech_list: list[Tech], model: str) -> list[Tech]:
```

```
    _search_history.append(model)
```

```
    return [tech for tech in tech_list if tech.model.lower() == model.lower()]
```

```
def create_tech_index(tech_list: list[Tech]) -> dict[str, Tech]:
```

```
    return {tech.inventory_number: tech for tech in tech_list}
```

```
def get_search_history() -> tuple[str, ...]:
```

```
    return tuple(_search_history)
```

```
def sort_by_vendor(tech_list: list[Tech]) -> list[Tech]:
```

```
    return sorted(tech_list, key=lambda tech: tech.vendor.lower())
```

```

def sort_items(
    tech_list: list[Tech],
    key: Callable[[Tech], object],
    reverse: bool = False,
) -> list[Tech]:
    return sorted(tech_list, key=key, reverse=reverse)

def filter_by_price_range(tech_list: list[Tech], min_price: float, max_price: float) -> list[Tech]:
    return [tech for tech in tech_list if min_price <= tech.price <= max_price]

def filter_items(tech_list: list[Tech], predicate: Callable[[Tech], bool]) -> list[Tech]:
    return [tech for tech in tech_list if predicate(tech)]

def create_price_filter(min_price: float) -> Callable[[Tech], bool]:

    def predicate(tech: Tech) -> bool:
        return tech.price >= min_price

    return predicate

def create_type_filter(tech_type: str) -> Callable[[Tech], bool]:

    def predicate(tech: Tech) -> bool:
        return tech.type.lower() == tech_type.lower()

    return predicate

def get_unique_vendors(tech_list: list[Tech]) -> set[str]:

```

```
return {tech.vendor for tech in tech_list}
```

```
def get_unique_types(tech_list: list[Tech]) -> set[str]:
```

```
    return {tech.type for tech in tech_list}
```

```
def group_by_type(tech_list: list[Tech]) -> dict[str, list[Tech]]:
```

```
    grouped: defaultdict[str, list[Tech]] = defaultdict(list)
```

```
    for tech in tech_list:
```

```
        grouped[tech.type].append(tech)
```

```
    return dict(grouped)
```

```
def group_by_vendor_and_type(tech_list: list[Tech]) -> dict[str, dict[str, list[Tech]]]:
```

```
    grouped: defaultdict[str, defaultdict[str, list[Tech]]] = defaultdict(lambda:
defaultdict(list))
```

```
    for tech in tech_list:
```

```
        grouped[tech.vendor][tech.type].append(tech)
```

```
    return {vendor: dict(types_dict) for vendor, types_dict in grouped.items()}
```

```
def count_by_type(tech_list: list[Tech]) -> Counter:
```

```
    return Counter(tech.type for tech in tech_list)
```

```
def count_by_vendor(tech_list: list[Tech]) -> Counter:
```

```
    return Counter(tech.vendor for tech in tech_list)
```

```
@measure_time
```

```
def avg_price(tech_list: list[Tech]) -> float:
```

```
    if not tech_list:
```

```
        return 0.0
```

```
    total_price = sum(tech.price for tech in tech_list)
```

```
return total_price / len(tech_list)
```

```
def find_most_expensive_tech(tech_list: list[Tech]) -> Tech | None:
```

```
    if not tech_list:
```

```
        return None
```

```
    return max(tech_list, key=lambda tech: tech.price)
```

```
def find_cheapest_tech(tech_list: list[Tech]) -> Tech | None:
```

```
    if not tech_list:
```

```
        return None
```

```
    return min(tech_list, key=lambda tech: tech.price)
```

```
def price_of_all_tech(tech_list: list[Tech]) -> float:
```

```
    return sum(tech.price for tech in tech_list)
```

```
def average_of_values(*prices: float) -> float:
```

```
    if not prices:
```

```
        return 0.0
```

```
    return sum(prices) / len(prices)
```

```
def create_tech_record(**fields) -> dict:
```

```
    return dict(fields)
```

```
def process_pipeline(tech_list: list[Tech], *steps: Callable[[list[Tech]], list[Tech]]) -> list[Tech]:
```

```
    result = tech_list
```

```
    for step in steps:
```

```
        result = step(result)
```

```
    return result
```

```

def build_summary_report(tech_list: list[Tech]) -> dict:
    return {
        "total_items": len(tech_list),
        "total_value": price_of_all_tech(tech_list),
        "average_price": (sum(tech.price for tech in tech_list) / len(tech_list)) if tech_list
        else 0.0,
        "unique_vendors": get_unique_vendors(tech_list),
        "unique_types": get_unique_types(tech_list),
        "count_by_type": dict(count_by_type(tech_list)),
        "most_expensive": find_most_expensive_tech(tech_list),
    }

```

```

def benchmark_search(tech_list: list[Tech], inventory_number: str) -> dict[str, float]:
    start = perf_counter()
    for tech in tech_list:
        if tech.inventory_number == inventory_number:
            break
    list_time = perf_counter() - start

    index = create_tech_index(tech_list)
    start = perf_counter()
    index.get(inventory_number)
    dict_time = perf_counter() - start

    return {"list_search_seconds": list_time, "dict_search_seconds": dict_time}

```